Network Intrusion Detection System Using Machine Learning

BTech CSE (AI & ML) Minorr Project Report ·

KANAK GIHAR

Abstract

This project builds a defensive, machine-learning-based Network Intrusion Detection System (NIDS) that classifies network-flow records as *Normal* or *Potentially Malicious*. Using the UNSW-NB15 dataset, a reproducible scikit-learn pipeline trains a Logistic Regression baseline and a Random Forest classifier. The selected Random Forest reaches 93.17% accuracy, 95.93% recall and a 9.80% false-positive rate on a held-out test split. A Streamlit dashboard lets users upload flow records, view analytics and inspect feature importance.

1. Objective

The aim is an end-to-end, honest working system: data validation, leakage-free training, evaluation with class-wise metrics, a saved model, a prediction module, tests and a web dashboard. All reported numbers come from actual runs. The project is for defensive research and education only; no offensive capability is implemented.

2. Dataset

The data is UNSW-NB15 (Moustafa & Slay, UNSW Canberra, 2015), a synthetic testbed capture. The file used, `UNSW_NB15_testing-set.csv`, has 175,341 rows, 39 numeric flow features, three categorical features (`proto`, `service`, `state`), a binary `label` and an `attack_cat` field with ten categories (Normal plus nine attack types). There were no missing values, but **67,601 rows (39%) were duplicates** once the row `id` was ignored. They were removed before splitting so identical flows could not appear in both training and test data, leaving 107,740 records (55,850 malicious, 51,890 normal).

3. Methodology

1. **Validation and cleaning:** required columns, 0/1 target, label/category consistency, text stripping, removal of infinite values and duplicates.
2. **Leakage control:** `id`, `label` and `attack_cat` are never used as features.
3. **Split:** stratified 80/20 on `attack_cat` (seed 42): 86,192 training and 21,548 test records. The official pre-split pair was not available, so this split is documented rather than official.
4. **Preprocessing** (fitted on training data only): median imputation, signed log transform and standardisation for numeric features; most-frequent imputation and one-hot encoding for categorical features, with rare or unseen values grouped.
5. **Models:** class-weighted Logistic Regression and Random Forest (100 trees, depth 18).

6. **Selection:** 3-fold stratified cross-validation on the training split, scored by malicious-class F1. The test split was scored once, after selection.
7. **Artifacts:** pipelines saved with Joblib, with a metadata file holding the feature schema, library versions and SHA-256 hashes. The loader refuses tampered files, files outside the model folder and incompatible scikit-learn versions.

4. Results

Model (test split)	Accuracy	Precision	Recall	F1	False-positive rate	ROC AUC
Logistic Regression	91.02%	88.35%	95.23%	91.66%	13.52%	0.9739
Random Forest (selected)	**93.17%**	**91.33%**	**95.93%**	**93.57%**	**9.80%**	**0.9856**

Cross-validated F1 on the training split was 0.9160 for Logistic Regression and 0.9326 for Random Forest, so the selection did not depend on the test set. The Random Forest confusion matrix shows 9,361 normal flows correct, 1,017 false alarms, 455 missed attacks and 10,715 attacks detected. Per category, Reconnaissance (99.9%), DoS (99.6%), Generic (99.6%) and Exploits (99.5%) are classified almost perfectly, while Fuzzers (88.2%), Analysis (86.2%) and Normal traffic (90.2%) are weaker. Backdoor and Worms reach 100%, but their test counts are small (307 and 26).

The most influential features are `sttl`, `ct_state_ttl` and `dload`. These are statistical associations learned from this dataset, not causes of attacks, and they may partly reflect how the testbed was generated.

5. Dashboard and Testing

The Streamlit application has five sections: Overview, CSV Traffic Analysis (column validation, adjustable threshold, risk levels, downloadable predictions), Security Analytics, Model Explainability and Project Information. The attack score it shows is a model output, not a calibrated probability. A suite of 40 unit tests covers target validation, empty and malformed files, missing columns, missing values, unseen categories, leakage, artifact tampering and version mismatches, and all pass. The dashboard code has been syntax-checked but not yet run, because Streamlit was unavailable in the build environment. The project is prepared for GitHub and Streamlit Community Cloud, but it has not been deployed.

6. Limitations and Ethical Use

Results come from one synthetic 2015 dataset and one split, so they should not be expected to transfer to real networks or modern attacks. A false-positive rate near 10% would generate too many alerts in practice. Only flow-level features are used, with no payload inspection and no handling of concept drift. The system should be used only on traffic the user owns or is authorised to analyse, and its flags should guide human review, never automated punitive action.

7. Conclusion and Future Work

The project delivers a reproducible, tested ML pipeline and dashboard with measured, honestly reported performance. Future work includes evaluating on the official separate training and testing files, probability calibration, threshold tuning to reduce false alarms, gradient-boosted models, multi-class attack classification and validation on more recent datasets.

Reference

Moustafa, N., & Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. *Military Communications and Information Systems Conference (MilCIS)*, UNSW Canberra.